Indian Institute of Technology Kharagpur

Department of Artificial Intelligence

Deep Learning Assignment 3: Autoregressive Image Generation

Focus: PixelCNN and Sequential PixelRNN (Quantized MNIST)

Laboratory Instructions

- **Duration:** 3 Hours.
- **Environment:** Use a GPU-enabled environment (Colab or local) because the autoregressive sampling loop is sequential and slow on CPU.
- **Dataset:** MNIST (`torchvision.datasets.MNIST`, downloaded directly in the notebook).
- **Important Note:** Every deliverable requires only code, a computed numerical value, or a visualization. There are no written explanations required.

1. Loading Dataset [2 Marks]

- Load MNIST, display a 3×3 grid of sample images, and print the shape/pixel range of a raw image. *[2 Marks]*

2. Preprocessing [4 Marks]

- Resize all images to 16×16 using `transforms.Resize`. *[1 Mark]*
- Quantize pixel intensities into $K = 4$ levels: scale the $[0, 1]$ tensor by 3, round to the nearest integer, cast to `torch.long`. Display a few quantized images to confirm correctness. *[3 Marks]*

3. Data Preparation [4 Marks]

- Use the entire MNIST dataset — all 60,000 training images (90/10 train/val split) and all 10,000 test images — with a fixed random seed (42) for reproducibility. *[2 Marks]*
- Build DataLoaders for all three splits, batch size = 64. *[2 Marks]*

4. PixelCNN: Masked Convolutions [18 Marks]

4.1 Task 4.1: Masked Convolution Layer [8 Marks]

- Implement `MaskedConv2d` (inherits `nn.Conv2d`) with a mask type flag:
  - **Type A:** zeroes the center pixel and everything after it (first layer only).
  - **Type B:** keeps the center pixel, zeroes everything after it (all later layers).
- Apply the mask to the weights before every forward pass — either by multiplying a registered mask buffer directly into the weight tensor, or by computing a masked copy of the weights on the fly. Either standard implementation is acceptable.

4.2 Task 4.2: Architecture [4 Marks]

- 1 Type-A layer (kernel = 7, padding = 3, channels = 64), followed by a ReLU, then 5 Type-B layers (kernel = 3, padding = 1, channels = 64), each followed by a ReLU, and finally a 1×1 convolution mapping to $K = 4$ output logits (no activation after the final layer).
- Cast inputs to `float32` and scale to $[0, 1]$ (divide by 3) before the forward pass.

4.3 Task 4.3: Training [2 Marks]

- Train 10 epochs, `CrossEntropyLoss` (per pixel), Adam, lr=0.001, batch size=64.

4.4 Task 4.4: Loss Curves [1 Mark]

- Plot training and validation losses.

4.5 Task 4.5: Test Evaluation [1 Mark]

- Report test loss (average per-pixel negative log-likelihood, in nats) and bits-per-dimension (BPD), computed as BPD $=$ test_loss$/\ln(2)$.

4.6 Task 4.6: Autoregressive Generation [2 Marks]

- Starting from a blank canvas, sample pixels one at a time in raster-scan order (softmax + `torch.multinomial`, `manual_seed(42)`). Display a 5×5 grid of generated digits.

5. PixelRNN: Sequential LSTM [19 Marks]

Treat the 16×16 image as one flattened length-256 sequence (raster-scan order).

5.1 Task 5.1: Shifted Input [3 Marks]

- Build the causally-shifted input: position t's input is the true pixel value at position $t - 1$. Use a dedicated Start-of-Sequence token (value 4) at position 0 — this must not collide with any of the four quantized pixel values (0–3).

5.2 Task 5.2: Architecture [6 Marks]

- `nn.Embedding(5, 16)` $\rightarrow$ single-layer `nn.LSTM(16, 64, batch_first=True)` over the full sequence $\rightarrow$ `nn.Linear(64, 4)` per timestep.

5.3 Task 5.3: Training [2 Marks]

- Same loss/optimizer/lr/batch size as Section 4, 10 epochs.

5.4 Task 5.4: Loss Curves [1 Mark]

- Plot training and validation losses.

5.5 Task 5.5: Test Evaluation [2 Marks]

- Report test loss (average per-pixel negative log-likelihood, in nats) and bits-per-dimension (BPD), same definition as Section 4.5.

5.6 Task 5.6: Autoregressive Generation [5 Marks]

- Feed the SOS token as the first input, sample the first pixel from the resulting distribution, then feed each newly sampled pixel back in one step at a time (carrying the LSTM hidden state forward) until all 256 positions are filled. Reshape the resulting sequence to 16×16 and display a 5×5 grid of generated digits.

6. Comparing the Two Models [3 Marks]

- Tabulate bits-per-dimension (BPD) and training wall-clock time for both models. *[3 Marks]*

Total: 50 Marks